C++ Distributed Counters

Committee: ISO/IEC JTC1 SC22 WG21 SG1 Concurrency
Document Number: P0261r0
Date: 2016-02-14
Authors: Lawrence Crowl
Reply To: Lawrence Crowl, Lawrence@Crowl.org

Introduction

Counters are ubiquitous in computing. For multi-threaded programs, atomic integers can provide safe concurrent counting.

In many cases, counters are much more commmonly incremented than read. For example, long-running multithreaded server programs may maintain many counters to aid in program diagnosis. Counters are often read only in response to a network query or program termination.

Unfortunately, the cost incrementing an atomic integer is significantly greater than reading one, even when reads are rare relative to increments. As the program scales, the cost of incrementing rarely-read atomic integers can limit program performance.

We need a mechanism that minimizes the cost of incrementing a counter, even if it has increased costs to read the counter.

Cilk adder-reducers [Cilk-Reducer] are one such mechanism. Unfortunately, the reduction is control-dependent and hence not suitable for external inspection of the count.

We propose a counter that has a complexity similar to atomic integers in the simple case, but enables programmers to easily distribute that counter across multiple threads when scalability becomes a concern.

This proposal restricts itself to precise counters. Statistical counters, where the read count may be slightly different from the count operations, can produce better performance [Dice-Lev-Moir-2013].

Solution

We propose two coordinated counter types that collectively provide for distributed counting. The essential component of the design is a trade-off between the cost of incrementing the counter, the cost of loading (reading) the counter value, and the "lag" between an increment and its appearance in a "read" value. That is, the read of a counter value may not reflect the most recent increments. However, no count will be lost.

These counters are parameterized by the base integer type that maintains the count. Avoid situations that overflow the integer, as that may have undefined behavior. This constraint implies that counters must be sized to their use. (For dynamic mitigation, see the `exchange` operation below.)

The base of the design is the concept of a counter bumper. This concept only provides for increasing or decreasing a counter. None of its operations provide any means to read the count.

The first type programmers will encounter is the `general_counter`. This counter implements the bumper concept and provides two additional operations, `load` and `exchange`. The `load` operation returns the current count. Note that 'current' is a fuzzy notion

when other threads are concurrently incrementing the counter. The `exchange` operation replaces the current count with the given value and returns the old value. This operation has particular utility in long-running programs in occasionally draining the counter to prevent integer overflow.

```
general_counter<int> red_count;

void count_red( Bag bag ) {
    for ( auto i: bag )
        if ( i.is_red() )
            ++red_count;
}

int how_many_red() {
    return red_count.load();
}
```

When contention becomes a problem, programmers distribute the counting with a `counter_broker` objects. These will typically be thread-local objects, but could be CPU-local objects or even stack-based objects.

```
general_counter<int> red_count;
thread_local counter_broker<int> thread_red( red_count );

void count_red( Bag bag )
    for ( auto i: bag )
        if ( i.is_red() )
            ++thread_red;
}
```

The association of counter brokers to general counter variables is an essential part of the distribution of counters, and not easily moved or copied. Therefore, we provide no copy or move operations for these types.

Managing the association between general counters and their attached brokers can require a non-trivial amount of space and time. Therefore, we provide a means to handle many counters with one name with the `general_counter_array`. The size of the counter arrays is fixed at construction time. Counter arrays have the same structure as single-value counters, with the following exceptions.

- Increment a counter by first indexing into its array and then applying one of the bumper operations. E.g. `++ctr_ary[i]`.
- The load and exchange operations take an additional index parameter.

Implementation

Olivier Giroux suggested an implementation in which the set of attached brokers was tracked by an intrusive list through the brokers themselves, rather than through a separate set data structure. This approach has some advantages.

- No dynamic allocation is required.
- The constructor of the general counter can be constexpr, which avoids initialization order problems for global objects.
- The head of the list is null until such time as a broker attaches itself, which substantially reduces the cost of using a general counter in the case where there are no brokers.

An implementation of an earlier version of the design (with the separate broker set data structure) is available from the Google Concurrency Library https://github.com/alasdairmackintosh/google-concurrency-library at .../blob/master/include/counter.h. This paper's `general_counter` maps to `counter::strong_duplex` and this paper's `counter_broker` maps to `counter::strong_broker`.

Wording

The distributed counter definition is as follows.

?.? Distributed counters [distctr]

Add a new section.

?.?.1 General [distctr.general]

Add a new section.

This section provides mechanisms for distributed counters. These mechanisms ease the production of scalable concurrent programs.

?.?.2 Synopsis [distctr.syn]

Add a new section.

```
template< typename Integral > class general_counter;
template< typename Integral > class counter_broker;
template< typename Integral > class general_counter_array;
template< typename Integral > class counter_broker_array;
```

?.?.3 Bumper concept [distctr.bumper]

Add a new section.

The bumper concept provides a common interface for increasing or decreasing the value of a counter. It consists of the following operations.

`void operator +=( integer )`
`void operator -=( integer )`

Effects:

Atomically add/subtract a value to/from the counter. There is no default value.

Returns:

void. [*Note:* These operations specifically do not return the count value, which would defeat the purpose of optimizing for increments over loads. —*end note*]

Synchronization:

None. [*Note:* Detecting a count that you know indicates that an event has occured does not mean that any other memory updates associated with that event are visible to the current thread. Acting specially on any particular thread count requires additional synchronization. —*end note*]

`void operator ++()`
`void operator ++(int)`
`void operator --()`
`void operator --(int)`

Effects:

As if *counter*`+=1` or *counter*`-=1`, respectively.

?.?.4 Class template `general_counter` [distctr.genctr]

Add a new section.

```
namespace std {
template< typename Integral >
class general_counter
{
  general_counter( const general_counter& ) = delete;
  general_counter& operator=( const general_counter& ) = delete;
public:
  constexpr general_counter( Integral );
  constexpr general_counter();
  ~counter_broker();
  void operator +=( Integral );
  void operator -=( Integral );
  void operator ++();
```

```
  void operator ++(int);
  void operator --();
  void operator --(int);
  Integral load();
  Integral exchange( Integral to );
}
```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

The `general_counter` class template implements the bumper concept ([distctr.bumper]).

`constexpr general_counter( Integral )`
`constexpr general_counter()`

Effects:

Initialize the counter with the given value, or zero if no value given.

Synchronization:

None.

`~general_counter()`

Requirements:

All attached brokers shall have been previously destroyed.

Effects:

Destroy the counter.

Synchronization:

None.

`Integer load()`

Returns:

the value of the counter. [*Note:* The value will be an approximation if any threads may concurrently update the counter. —*end note*]

Synchronization:

None. [*Note:* Detecting a count that you know indicates that an event has occured does not mean that any other memory updates associated with that event are visible to the current thread. Acting specially on any particular thread count requires additional synchronization. —*end note*]

`Integer exchange( Integer )`

Effects:

Atomically replaces the value of the counter with the value of the parameter.

Returns:

The replaced value.

Synchronization:

None. [*Note:* Detecting a count that you know indicates that an event has occured does not mean that any other memory updates associated with that event are visible to the current thread. Acting specially on any particular thread count requires additional synchronization. —*end note*]

?.?.5 Class template `counter_broker` [distctr.broker]

Add a new section.

```
template< typename Integral >
class counter_broker
{
  counter_broker( const counter_broker& ) = delete;
  counter_broker& operator=( const counter_broker& ) = delete;
public:
  constexpr counter_broker( general_counter<Integral>& );
  ~counter_broker();
  void operator +=( Integral );
  void operator -=( Integral );
  void operator ++();
  void operator ++(int);
  void operator --();
  void operator --(int);
};
```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

The `counter_broker` class template implements the bumper concept ([distctr.bumper]).

`constexpr counter_broker( general_counter& )`

Effects:

> Initialize the broker, atomically associating it with the given counter.

Synchronization:

> None.

`~counter_broker()`

Effects:

> Atomically moves any count value retained within the broker to its associated counter. Atomically disassociates the broker from the counter, with respect to the counter. Destroy the broker.

Synchronization:

> None.

?.?.6 Class `general_counter_array` [distctr.generalarray]

Add a new section.

```
template< typename Integral >
class general_counter_array
{
  general_counter_array( const general_counter_array& ) = delete;
  general_counter_array& operator=( const general_counter_array& ) = delete;
public:
  general_counter_array( size_type size );
  some_bumper_type& operator[]( size_type idx );
  size_type size();
  Integral load( size_type idx );
  Integral exchange( size_type idx, Integral value );
};
```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

`constexpr general_counter_array( size_type );`

Effects:

> Initialize the counter array with the given number of counters.

Synchronization:

> None.

`~general_counter_array()`

Requirements:

All attached brokers shall have been previously destroyed.

Effects:

Destroy the counter.

Synchronization:

None.

some_bumper_type`& operator[]( size_type index )`

Returns:

A reference to an object, identified by the `index`, implementing the bumper concept [distctr.bumper].

Synchronization:

None.

`Integer load( size_type index )`

Returns:

the value of the counter with the given `index`. [*Note:* The value will be an approximation if any threads may concurrently update the counter. —*end note*]

Synchronization:

None. [*Note:* Detecting a count that you know indicates that an event has occured does not mean that any other memory updates associated with that event are visible to the current thread. Acting specially on any particular thread count requires additional synchronization. —*end note*]

`Integer exchange( size_type index, Integer value )`

Effects:

Atomically replaces the value of the counter at the given `index` with the given `value`.

Returns:

The replaced value.

Synchronization:

None. [*Note:* Detecting a count that you know indicates that an event has occured does not mean that any other memory updates associated with that event are visible to the current thread. Acting specially on any particular thread count requires additional synchronization. —*end note*]

?.?.7 Class `counter_broker_array` [distctr.brokerarray]

Add a new section.

```
template< typename Integral >
class counter_broker_array
{
  counter_broker_array( const counter_broker_array& ) = delete;
  counter_broker_array& operator=( const counter_broker_array& ) = delete;
public:
  counter_broker_array( size_type size );
  some_bumper_type& operator[]( size_type idx );
  size_type size();
};
```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5

[atomics.types.generic]).

`constexpr counter_broker_array( size_type );`

Effects:

Initialize the broker array. Atomically associate the broker array with the counter array with respect to the array.

Synchronization:

None.

`~counter_broker_array()`

Requirements:

All attached brokers shall have been previously destroyed.

Effects:

Atomically moves any count values retained within the broker to its associated counter array. Atomically disassociateds the broker array from the counter array, with respect to the counter array. Destroy the broker array.

Synchronization:

None.

`some_bumper_type``& operator[]( size_type index )`

Returns:

A reference to an object, identified by the `index`, implementing the bumper concept [distctr.bumper].

Synchronization:

None.

Revision History

This paper revises ISO/IEC JTC1 SC22 WG21 N3706 - 2013-09-01. Its changes are as follows:

- Add more context to the introduction and rework subsequent discussion.
- Reduce the number of types to four, a `general_counter` equivalent to the earlier `strong_duplex`, a `counter_broker` equivalent to the earlier `strong_broker`, a `general_counter_array` equivalent to the earlier `strong_duplex_array`, and a `counter_broker_array` equivalent to the earlier `strong_broker_array`.
- Remove the atomicity specification. Only full and relaxed atomicity is now supported.
- As a consequence of the above two points, drastically reduce the solution presentation.
- Update the implementation discussion with a new technique and new pointers to the code base.
- Add proposed wording.

N3706 revised ISO/IEC JTC1 SC22 WG21 N3355 = 12-0045 - 2012-01-14. Its changes are as follows:

- Add more context to the introduction.
- Change named `inc` and `dec` functions to operators.
- Place all declarations within namespace `counter`. The namespace serves to avoid redundancy in names.
- Rename types for increased clarity.
- Change separate serial and atomic counter types with a single type templatized on the atomicity.

- Add issues of memory order and contention to the atomicity parameter.
- Add counter arrays.
- Add open issues section.
- Add implementation section.
- Add revision history section.
- Add references section.

References

[Cilk-Reducer]
"Reducers", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/cref_cls/common/cilk_bk_reducer_intro.htm; "Intel Cilk Plus", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/cref_cls/common/cilk_bk_using_cilk.htm; "Intel C++ Compiler XE 12.1 User and Reference Guides", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/main/main_cover_title.htm

[Dice-Lev-Moir-2013]
"Scalable Statstics Counters"; Dave Dice, Yossi Lev, Mark Moir; SPAA'13, June 23-25, 2013, Montrèal Quèbec, Canada.

[Lev-Moir-2011]
"Lightweight Parallel Accumulators Using C++ Templates"; Yossi Lev, Mark Moir; ICSE'11, May 21-28, 2011, Waikiki, Honolulu, HI, USA